

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КАЛЬКУЛЯТОР УРОВНЯ СУПЕРГЕРОЯ»
Вариант 27

Студент:

А.А. Фалько

Преподаватель:

О.М. Внук

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки IntelliJ IDEA при создании объектно-ориентированного приложения на языке Java, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 27 необходимо разработать калькулятор уровня супергероя. Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект Java в IntelliJ IDEA, изучить средства редактирования, сборки, запуска и отладки;
2. Подобрать средства IntelliJ IDEA для работы с UML и SysML v2 и отразить результат настройки в отчёте;
3. Разработать графическое приложение с главной формой и отдельным окном ввода роста, веса, возраста, количества подтягиваний и продолжительности сна;
4. Обеспечить определение уровня супергероя, сохранение последних корректных данных и их восстановление при повторном вводе;
5. Реализовать проверку входных данных и вывод понятного сообщения при ошибке;
6. Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью;
7. Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки объединяет редактор исходного кода, средства навигации и рефакторинга, систему сборки, запуск приложения, отладчик, терминал и интеграцию с системой контроля версий. IntelliJ IDEA анализирует Java-код во время редактирования, показывает ошибки и предупреждения, предлагает быстрые исправления и позволяет запускать класс с методом `main` непосредственно из редактора.

2.2 Редактирование и навигация в IntelliJ IDEA

Редактор IntelliJ IDEA поддерживает автодополнение, импорт классов, форматирование, безопасное переименование и поиск использований. Для перехода к объявлению используется `Ctrl+B`, для поиска использований – `Alt+F7`, для форматирования – `Ctrl+Alt+L`. Структура проекта отображается в окне `Project`, сообщения компилятора и программы – в окнах `Build` и `Run`.

2.3 Сборка, запуск и отладка Java-приложения

Исходные файлы компилируются компилятором `javac` в байт-код JVM. В IntelliJ IDEA проект собирается командой `Build Project`, а конфигурация запуска класса `Main` выполняется командами `Run` и `Debug`. Точка останова устанавливается щелчком в поле слева от строки или сочетанием `Ctrl+F8`. Во время остановки окно `Debug` показывает стек вызовов и значения переменных. Команды `Step Over`, `Step Into` и `Step Out` позволяют выполнять программу пошагово.

2.4 Паттерн MVC

MVC разделяет приложение на три части. `Model` хранит состояние и выполняет предметную логику. `View` формирует интерфейс и отображает данные. `Controller` обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сообщает подписчикам об изменении состояния. В разработанном приложении `SuperHeroModel` использует паттерн «Наблюдатель» (`Observer`): `PropertyChangeSupport` рассылает события слушателям `PropertyChangeListener`. Главное окно подписывается на модель в методе `observe` и обновляет отображение при получении события.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработан калькулятор уровня супергероя, который принимает рост в сантиметрах, вес в килограммах, возраст в полных годах, количество подтягиваний и продолжительность сна за сутки. Программа определяет одну из четырёх категорий: «Стажёр», «Герой», «Легенда» или «Нужно срочно в отпуск». На главной форме размещена кнопка «Ввести данные», открывающая модальное окно `InputDialog`. После расчёта главное окно отображает введённые параметры, уровень, баллы и ИМТ. При повторном открытии формы восстанавливаются последние корректные значения.

3.2 Настройка среды и расширений для UML и SysML v2

В IntelliJ IDEA открыт проект `SuperHeroCalc` с каталогом исходного кода `src` и пакетами `superhero.model`, `superhero.view` и `superhero.controller`. В качестве Project SDK выбран `openjdk-26`. Класс `Main` содержит метод `main` и используется в конфигурации запуска. Для UML применяется плагин `PlantUML Integration` и файл `theory/mvc-class-diagram.puml`. Для материалов SysML v2 используется текстовая нотация языка и доступное совместимое средство подсветки или моделирования, установленное через `Settings – Plugins`.

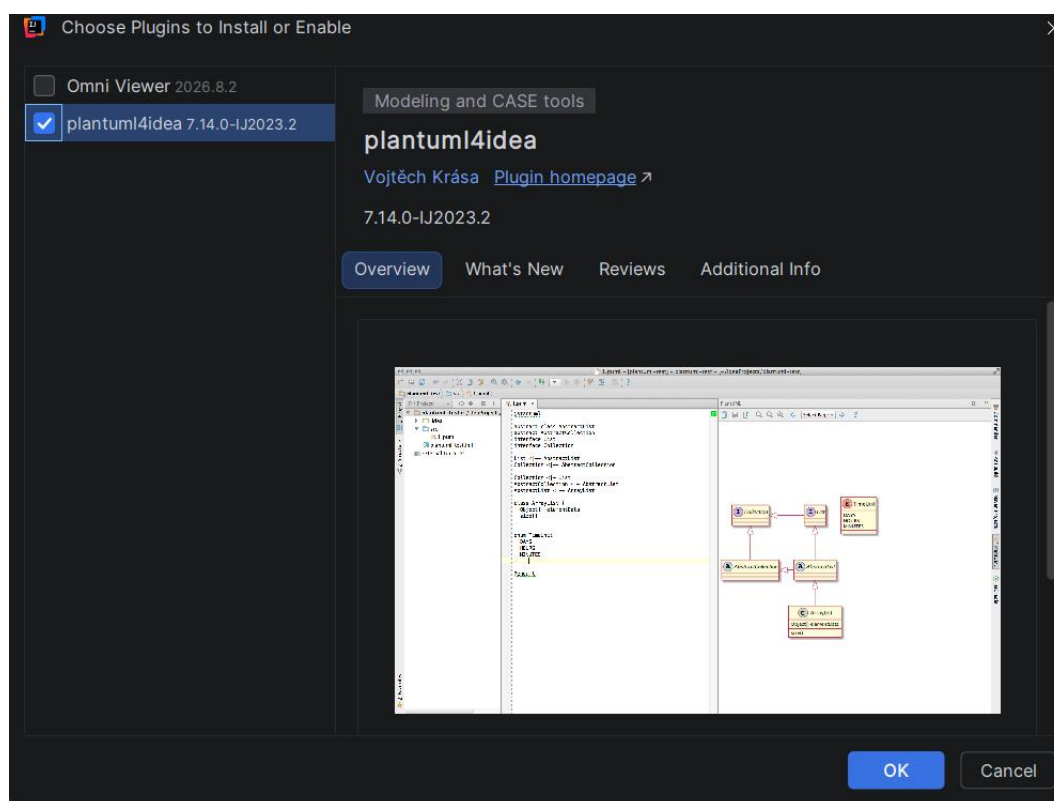


Рисунок 3.1 – Средства IntelliJ IDEA для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2:

```
package SuperHeroCalculator {
    part def SuperHeroModel;
    part def MainFrame;
    part def InputDialog;
    part def SuperHeroController;

    part application {
        part model : SuperHeroModel;
        part view : MainFrame;
        part dialog : InputDialog;
        part controller : SuperHeroController;
        connect view to controller;
        connect controller to model;
        connect model to view;
    }
}
```

3.3 Демонстрация отладчика

Для демонстрации отладчика используется метод `evaluate` класса `SuperHeroModel`. Точку останова можно установить на вычислении `bmi` или `score`. В окне `Debug` прослеживаются входные значения `data`, перевод роста в метры, баллы за подтягивания `pullUpScore`, итоговая сумма `score` и выбранный `level`. Дополнительные точки в `calculate` перед `firePropertyChange` и в `MainFrame.showState` позволяют проследить уведомление представления активной моделью.

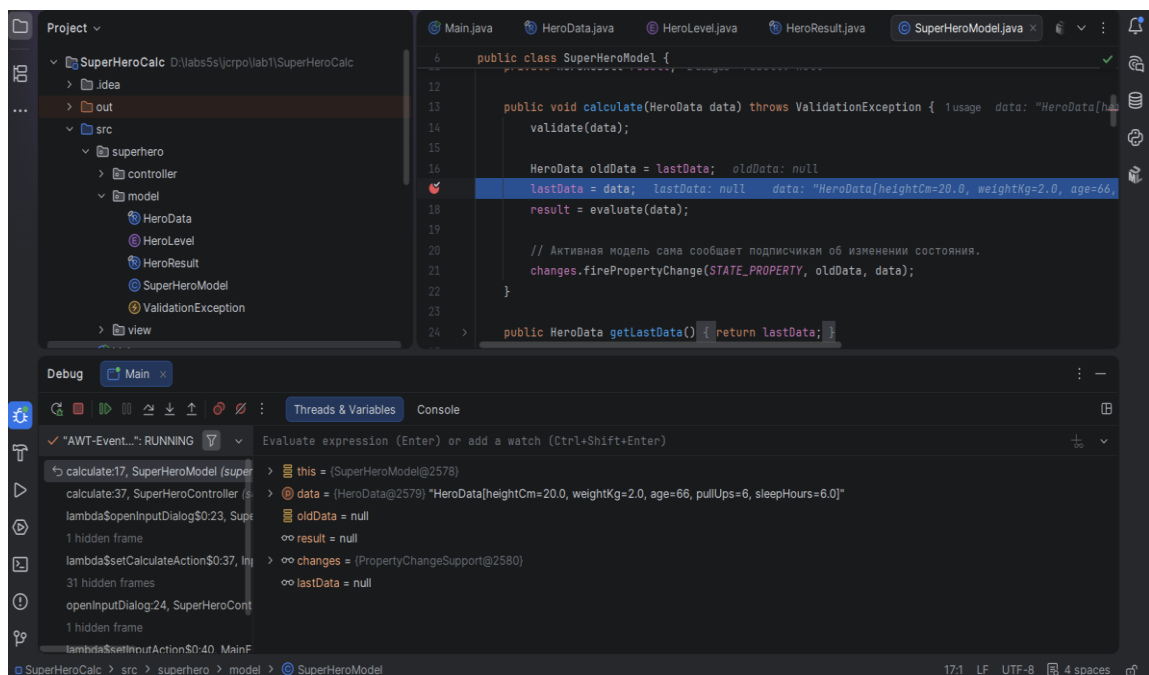


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- SuperHeroModel – хранит последние корректные данные HeroData и результат HeroResult, проверяет значения, рассчитывает уровень и уведомляет слушателей через PropertyChangeListener;
- MainFrame – главное окно; в методе observe регистрирует слушатель PropertyChangeListener и при изменении модели обновляет надписи методом showState;
- InputDialog – модальное окно для ввода пяти параметров, восстановления значений и отображения ошибок;
- SuperHeroController – обрабатывает действия пользователя, преобразует строки в числа, создаёт HeroData и передаёт его модели;
- Main – точка входа, создающая компоненты в потоке обработки событий Swing;
- HeroData и HeroResult – неизменяемые записи данных, HeroLevel – перечисление уровней, ValidationException – исключение для сообщения об ошибке ввода.

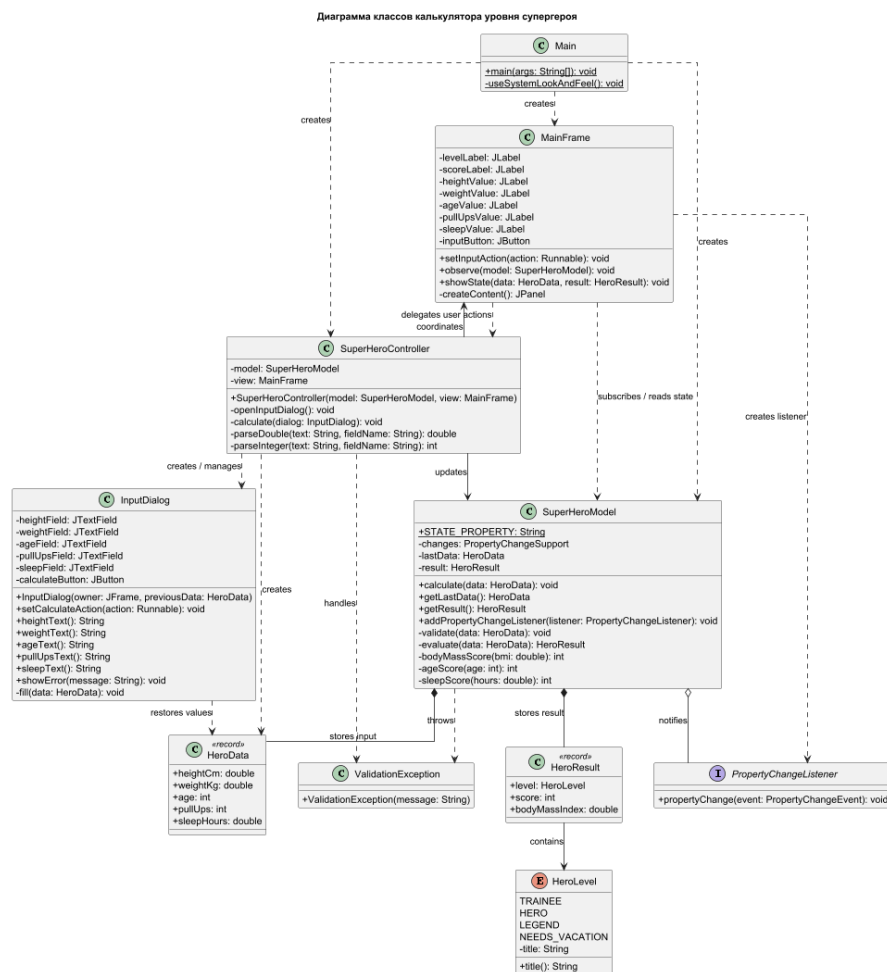


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

SuperHeroModel реализует активную модель. Метод `calculate` сначала вызывает `validate`: рост, вес и возраст должны быть положительными, подтягивания – неотрицательными, сон – от 0 до 24 часов. Для вещественных значений исключаются NaN и бесконечность. После проверки `evaluate` вычисляет ИМТ как отношение веса к квадрату роста в метрах, сумму баллов и уровень. Поскольку методичка не задаёт формулу, используется условная учебная система оценки. За ИМТ от 18,5 включительно до 25 начисляется 25 баллов; от 16 до 18,5 и от 25 до 30 – 15; в остальных случаях – 5. За каждое подтягивание начисляется 4 балла, максимум 40. Возраст 19–35 лет даёт 20 баллов, 36–50 лет – 15, остальные допустимые значения – 10. Сон 7–9 часов даёт 20 баллов, оставшиеся значения в диапазоне 6–10 – 12, в диапазоне 5–11 – 5, за пределами – 0. При сне меньше 5 или больше 11 часов выбирается «Нужно срочно в отпуск». Иначе от 80 баллов присваивается «Легенда», от 55 – «Герой», ниже 55 – «Стажёр». После сохранения результата `firePropertyChange` публикует событие изменения входных данных. При полностью одинаковых данных повторное событие не отправляется, поскольку состояние не изменилось.

3.6 Представление

MainFrame отображает последние введённые параметры, уровень супергероя, сумму баллов и ИМТ. Метод `observe` подписывает главное окно на изменения модели; обработчик события читает `getLastData` и `getResult`, затем вызывает `showState`. InputDialog содержит пять текстовых полей, кнопку «Рассчитать», назначенную кнопкой по умолчанию для Enter, и кнопку «Отмена».

3.7 Контроллер

SuperHeroController связывает модель с представлением и открывает InputDialog. Методы `parseDouble` и `parseInteger` преобразуют текстовые значения в числа; десятичная запятая заменяется точкой, а пробелы по краям удаляются. Контроллер создаёт HeroData и вызывает `model.calculate`. При успехе диалог закрывается. Ошибки разбора преобразуются в `ValidationException`; это же исключение используется моделью для недопустимых значений. Контроллер перехватывает его и вызывает `dialog.showError`, который выводит сообщение через `JOptionPane`.

3.8 Запоминание данных и обработка ошибок

Приложение отклоняет пустые поля, нечисловой ввод и значения, нарушающие правила модели. Например, при сне 25 часов показывается

сообщение о допустимом диапазоне от 0 до 24. Проверка выполняется до изменения сохранённых данных, поэтому при ошибке предыдущее корректное состояние остаётся прежним. Перед показом формы контроллер передаёт `model.getLastData` в конструктор `InputDialog`. Данные сохраняются в памяти до завершения программы. «Отмена» закрывает окно без изменения модели.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано Java-приложение варианта 27. Программа определяет уровень супергероя, сохраняет последнее корректное состояние и сообщает об ошибках ввода. Архитектура соответствует MVC с активной моделью: контроллер передаёт данные, SuperHeroModel выполняет расчёт и уведомляет представление через PropertyChangedSupport. Контрольные примеры по реализованной формуле: рост 180 см, вес 75 кг, возраст 25 лет, 10 подтягиваний и 8 часов сна дают 105 баллов и уровень «Легенда»; при 2 подтягиваниях – 73 балла и «Герой». Для роста 180 см, веса 120 кг, возраста 60 лет, 0 подтягиваний и 5,5 часа сна ожидаются 20 баллов и «Стажёр». При сне 4 часа выбирается «Нужно срочно в отпуск».

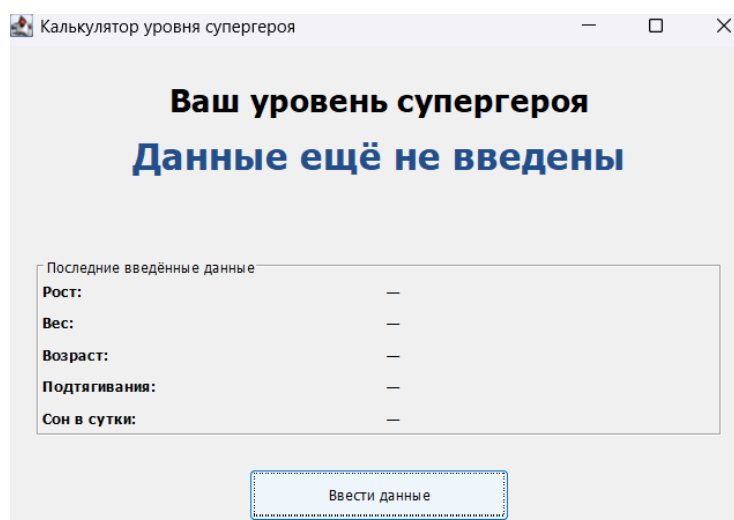


Рисунок 4.1 – Главная форма после запуска

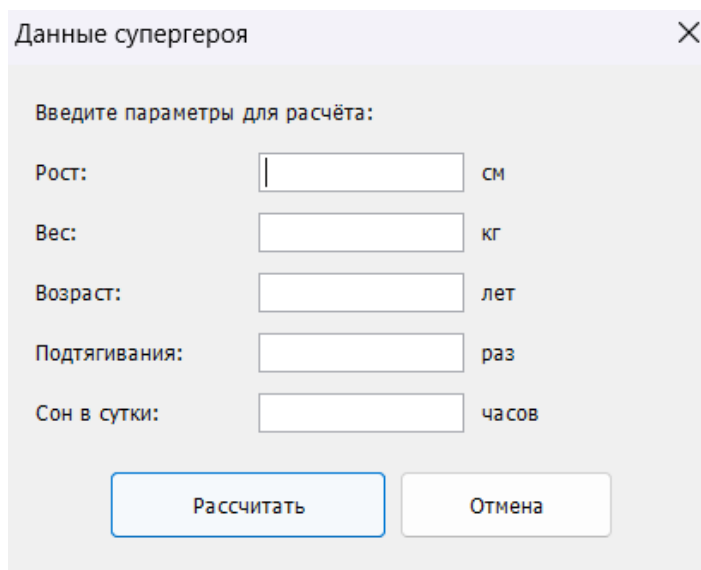


Рисунок 4.2 – Окно ввода данных

Калькулятор уровня супергероя

Ваш уровень супергероя

Легенда

Результат: 85 баллов · ИМТ: 154,3

Последние введённые данные

Рост:	180 см
Вес:	500 кг
Возраст:	20 лет
Подтягивания:	15000000 раз
Сон в сутки:	8 ч

Ввести данные

Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений. В IntelliJ IDEA основные рабочие окна называются Project, Build, Run, Debug, Problems и Terminal.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, быстрые исправления, автоматический импорт, форматирование, переход к объявлению, поиск использований, переименование, просмотр иерархии классов и анализ предупреждений. Эти операции позволяют изменять код быстрее и уменьшают вероятность механических ошибок.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

IntelliJ IDEA компилирует Java-файлы командой Build Project с использованием настроенного JDK. Для запуска выбирается класс Main с методом `public static void main(String[] args)` и выполняется команда Run. JVM загружает скомпилированные классы и вызывает точку входа. При Debug программа работает под управлением отладчика и останавливается на заданных точках.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет SuperHeroModel вместе с HeroData, HeroResult и HeroLevel, роль представления – MainFrame и InputDialog, а роль контроллера – SuperHeroController.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому после её изменения контроллер должен отдельно обновить интерфейс. Активная модель публикует событие при изменении состояния. В приложении SuperHeroModel вызывает `firePropertyChange`, а слушатель, зарегистрированный в `MainFrame.observe`, получает событие, читает новое состояние и вызывает `showState`. Контроллер не выполняет обновление надписей после расчёта.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности IntelliJ IDEA для редактирования, сборки, запуска и отладки Java-приложения. Рассмотрены средства работы с UML и SysML v2 и подготовлена диаграмма классов приложения.

Для варианта 27 разработан калькулятор уровня супергероя с графическим интерфейсом Swing. Реализованы ввод пяти параметров, расчёт одной из четырёх категорий, проверка значений и восстановление последних корректных данных. Компоненты разделены по MVC, а активная модель уведомляет представление об изменении через PropertyChangeSupport и PropertyChangeListener. Приложение использует стандартную библиотеку Java и не требует сторонних GUI-библиотек.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

Main.java

```
import superhero.controller.SuperHeroController;
import superhero.model.SuperHeroModel;
import superhero.view.MainFrame;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            useSystemLookAndFeel();

            SuperHeroModel model = new SuperHeroModel();
            MainFrame view = new MainFrame();
            new SuperHeroController(model, view);
            view.setVisible(true);
        });
    }

    private static void useSystemLookAndFeel() {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {
            // Если системная тема недоступна, Swing использует
            стандартную.
        }
    }
}
```

superhero/controller/SuperHeroController.java

```
package superhero.controller;

import superhero.model.HeroData;
import superhero.model.SuperHeroModel;
import superhero.model.ValidationException;
import superhero.view.InputDialog;
import superhero.view.MainFrame;

public class SuperHeroController {
    private final SuperHeroModel model;
    private final MainFrame view;
```

```

public SuperHeroController(SuperHeroModel model, MainFrame view) {
    this.model = model;
    this.view = view;

    view.setInputAction(this::openInputDialog);
    view.observe(model);
}

private void openInputDialog() {
    InputDialog dialog = new InputDialog(view, model.getLastData());
    dialog.setCalculateAction(() -> calculate(dialog));
    dialog.setVisible(true);
}

private void calculate(InputDialog dialog) {
    try {
        HeroData data = new HeroData(
            parseDouble(dialog.heightText(), "рост"),
            parseDouble(dialog.weightText(), "вес"),
            parseInteger(dialog.ageText(), "возраст"),
            parseInteger(dialog.pullUpsText(), "количество
подтягиваний"),
            parseDouble(dialog.sleepText(), "продолжительность
сна")
        );

        model.calculate(data);
        dialog.dispose();
    } catch (ValidationException exception) {
        dialog.showError(exception.getMessage());
    }
}

private double parseDouble(String text, String fieldName) throws
ValidationException {
    String normalized = text.trim().replace(',', '.');
    if (normalized.isEmpty()) {
        throw new ValidationException("Заполните поле «" + fieldName
+ "».");
    }
    try {
        return Double.parseDouble(normalized);
    } catch (NumberFormatException exception) {
        throw new ValidationException("Поле «" + fieldName + "»
должно содержать число.");
    }
}

```

```

        private int parseInteger(String text, String fieldName) throws
ValidationException {
            String normalized = text.trim();
            if (normalized.isEmpty()) {
                throw new ValidationException("Заполните поле «" + fieldName
+ "».");
            }
            try {
                return Integer.parseInt(normalized);
            } catch (NumberFormatException exception) {
                throw new ValidationException("Поле «" + fieldName + "»
должно содержать целое число.");
            }
        }
    }
}

```

superhero/model/HeroData.java

```

package superhero.model;

public record HeroData(
    double heightCm,
    double weightKg,
    int age,
    int pullUps,
    double sleepHours
) {
}

```

superhero/model/HeroLevel.java

```

package superhero.model;

public enum HeroLevel {
    TRAINEE("Стажёр"),
    HERO("Герой"),
    LEGEND("Легенда"),
    NEEDS_VACATION("Нужно срочно в отпуск");

    private final String title;

    HeroLevel(String title) {
        this.title = title;
    }

    public String title() {
        return title;
    }
}

```

superhero/model/HeroResult.java

```
package superhero.model;

    public record HeroResult(HeroLevel level, int score, double
bodyMassIndex) {
        }
    }
```

superhero/model/SuperHeroModel.java

```
package superhero.model;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class SuperHeroModel {
    public static final String STATE_PROPERTY = "state";

    private final PropertyChangeSupport changes = new
PropertyChangeSupport(this);
    private HeroData lastData;
    private HeroResult result;

    public void calculate(HeroData data) throws ValidationException {
        validate(data);

        HeroData oldData = lastData;
        lastData = data;
        result = evaluate(data);

        // Активная модель сама сообщает подписчикам об изменении
состояния.
        changes.firePropertyChange(STATE_PROPERTY, oldData, data);
    }

    public HeroData getLastData() {
        return lastData;
    }

    public HeroResult getResult() {
        return result;
    }

    public void addPropertyChangeListener(PropertyChangeListener
listener) {
        changes.addPropertyChangeListener(listener);
    }
}
```

```

        private void validate(HeroData data) throws ValidationException {
            if (!Double.isFinite(data.heightCm()) || data.heightCm() <= 0) {
                throw new ValidationException("Рост должен быть
положительным числом.");
            }
            if (!Double.isFinite(data.weightKg()) || data.weightKg() <= 0) {
                throw new ValidationException("Вес должен быть положительным
числом.");
            }
            if (data.age() <= 0) {
                throw new ValidationException("Возраст должен быть
положительным целым числом.");
            }
            if (data.pullUps() < 0) {
                throw new ValidationException("Количество подтягиваний
должно быть неотрицательным целым числом.");
            }
            if (!Double.isFinite(data.sleepHours()) || data.sleepHours() < 0
|| data.sleepHours() > 24) {
                throw new ValidationException("Продолжительность сна должна
быть числом от 0 до 24 часов.");
            }
        }

        private HeroResult evaluate(HeroData data) {
            double heightMeters = data.heightCm() / 100.0;
            double bmi = data.weightKg() / (heightMeters * heightMeters);

            int pullUpScore = (int) Math.min(40L, (long) data.pullUps() *
4L);

            int score = bodyMassScore(bmi)
                + pullUpScore
                + ageScore(data.age())
                + sleepScore(data.sleepHours());

            HeroLevel level;
            if (data.sleepHours() < 5 || data.sleepHours() > 11) {
                level = HeroLevel.NEEDS_VACATION;
            } else if (score >= 80) {
                level = HeroLevel.LEGEND;
            } else if (score >= 55) {
                level = HeroLevel.HERO;
            } else {
                level = HeroLevel.TRAINEE;
            }

            return new HeroResult(level, score, bmi);
        }

```

```

private int bodyMassScore(double bmi) {
    if (bmi >= 18.5 && bmi < 25) {
        return 25;
    }
    if (bmi >= 16 && bmi < 30) {
        return 15;
    }
    return 5;
}

private int ageScore(int age) {
    if (age >= 19 && age <= 35) {
        return 20;
    }
    if (age >= 36 && age <= 50) {
        return 15;
    }
    return 10;
}

private int sleepScore(double hours) {
    if (hours >= 7 && hours <= 9) {
        return 20;
    }
    if (hours >= 6 && hours <= 10) {
        return 12;
    }
    if (hours >= 5 && hours <= 11) {
        return 5;
    }
    return 0;
}
}

```

superhero/model/ValidationException.java

```

package superhero.model;

public class ValidationException extends Exception {
    public ValidationException(String message) {
        super(message);
    }
}

```

superhero/view/InputDialog.java

```

package superhero.view;

```

```

import superhero.model.HeroData;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JDialog;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTextField;
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.GridLayout;

public class InputDialog extends JDialog {
    private final JTextField heightField = new JTextField(10);
    private final JTextField weightField = new JTextField(10);
    private final JTextField ageField = new JTextField(10);
    private final JTextField pullUpsField = new JTextField(10);
    private final JTextField sleepField = new JTextField(10);
    private final JButton calculateButton = new JButton("Рассчитать");

    public InputDialog(JFrame owner, HeroData previousData) {
        super(owner, "Данные супергероя", true);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);
        setContentPane(createContent());
        fill(previousData);
        getRootPane().setDefaultButton(calculateButton);
        pack();
        setLocationRelativeTo(owner);
    }

    public void setCalculateAction(Runnable action) {
        calculateButton.addActionListener(event -> action.run());
    }

    public String heightText() {
        return heightField.getText();
    }

    public String weightText() {
        return weightField.getText();
    }

    public String ageText() {
        return ageField.getText();
    }

```

```

    }

    public String pullUpsText() {
        return pullUpsField.getText();
    }

    public String sleepText() {
        return sleepField.getText();
    }

    public void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Ошибка ввода",
JOptionPane.ERROR_MESSAGE);
    }

    private JPanel createContent() {
        JPanel root = new JPanel(new BorderLayout(12, 12));
        root.setBorder(BorderFactory.createEmptyBorder(16, 18, 14, 18));

        JPanel fields = new JPanel(new GridLayout(0, 3, 8, 9));
        addField(fields, "Рост", heightField, "см");
        addField(fields, "Вес", weightField, "кг");
        addField(fields, "Возраст", ageField, "лет");
        addField(fields, "Подтягивания", pullUpsField, "раз");
        addField(fields, "Сон в сутки", sleepField, "часов");

        JButton cancelButton = new JButton("Отмена");
        cancelButton.addActionListener(event -> dispose());

        JPanel buttons = new JPanel();
        calculateButton.setPreferredSize(new Dimension(130, 32));
        cancelButton.setPreferredSize(new Dimension(100, 32));
        buttons.add(calculateButton);
        buttons.add(cancelButton);

        root.add(new JLabel("Введите параметры для расчёта:"),
BorderLayout.NORTH);
        root.add(fields, BorderLayout.CENTER);
        root.add(buttons, BorderLayout.SOUTH);
        return root;
    }

    private void addField(JPanel panel, String label, JTextField field,
String unit) {
        panel.add(new JLabel(label + ":"));
        panel.add(field);
        panel.add(new JLabel(unit));
    }

```

```

private void fill(HeroData data) {
    if (data == null) {
        return;
    }
    heightField.setText(format(data.heightCm()));
    weightField.setText(format(data.weightKg()));
    ageField.setText(Integer.toString(data.age()));
    pullUpsField.setText(Integer.toString(data.pullUps()));
    sleepField.setText(format(data.sleepHours()));
}

private String format(double value) {
    return value == Math rint(value) ? String.format("%.0f", value)
: String.format("%.1f", value);
}
}

```

superhero/view/MainFrame.java

```

package superhero.view;

import superhero.model.HeroData;
import superhero.model.HeroResult;
import superhero.model.SuperHeroModel;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingConstants;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridLayout;

public class MainFrame extends JFrame {
    private final JLabel levelLabel = new JLabel("Данные ещё не
введены", SwingConstants.CENTER);
    private final JLabel scoreLabel = new JLabel(" ",
SwingConstants.CENTER);
    private final JLabel heightValue = new JLabel("-");
    private final JLabel weightValue = new JLabel("-");
    private final JLabel ageValue = new JLabel("-");
    private final JLabel pullUpsValue = new JLabel("-");
    private final JLabel sleepValue = new JLabel("-");
}

```

```

private final JButton inputButton = new JButton("Ввести данные");

public MainFrame() {
    super("Калькулятор уровня супергероя");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setMinimumSize(new Dimension(540, 390));
    setLocationByPlatform(true);
    setContentPane(createContent());
    pack();
    setLocationRelativeTo(null);
}

public void setInputAction(Runnable action) {
    inputButton.addActionListener(event -> action.run());
}

public void observe(SuperHeroModel model) {
    model.addPropertyChangeListener(event -> {
        if
(SuperHeroModel.STATE_PROPERTY.equals(event.getPropertyName())) {
            showState(model.getLastData(), model.getResult());
        }
    });
}

public void showState(HeroData data, HeroResult result) {
    heightValue.setText(format(data.heightCm()) + " см");
    weightValue.setText(format(data.weightKg()) + " кг");
    ageValue.setText(data.age() + " лет");
    pullUpsValue.setText(data.pullUps() + " раз");
    sleepValue.setText(format(data.sleepHours()) + " ч");

    levelLabel.setText(result.level().title());
    scoreLabel.setText(String.format("Результат: %d баллов · ИМТ:
%.1f", result.score(), result.bodyMassIndex()));
}

private JPanel createContent() {
    JPanel root = new JPanel(new BorderLayout(16, 16));
    root.setBorder(BorderFactory.createEmptyBorder(20, 24, 20, 24));

    JLabel title = new JLabel("Ваш уровень супергероя",
SwingConstants.CENTER);
    title.setFont(title.getFont().deriveFont(Font.BOLD, 22f));

    levelLabel.setFont(levelLabel.getFont().deriveFont(Font.BOLD,
26f));
    levelLabel.setForeground(new Color(35, 78, 140));

```

```

        JPanel heading = new JPanel(new GridLayout(0, 1, 0, 7));
        heading.add(title);
        heading.add(levelLabel);
        heading.add(scoreLabel);

        JPanel values = new JPanel(new GridLayout(0, 2, 10, 8));
        values.setBorder(BorderFactory.createTitledBorder("Последние
введённые данные"));
        addRow(values, "Пост:", heightValue);
        addRow(values, "Вес:", weightValue);
        addRow(values, "Возраст:", ageValue);
        addRow(values, "Подтягивания:", pullUpsValue);
        addRow(values, "Сон в сутки:", sleepValue);

        JPanel buttonPanel = new JPanel();
        inputButton.setPreferredSize(new Dimension(180, 36));
        buttonPanel.add(inputButton);

        root.add(heading, BorderLayout.NORTH);
        root.add(values, BorderLayout.CENTER);
        root.add(buttonPanel, BorderLayout.SOUTH);
        return root;
    }

    private void addRow(JPanel panel, String name, JLabel value) {
        JLabel label = new JLabel(name);
        label.setFont(label.getFont().deriveFont(Font.BOLD));
        panel.add(label);
        panel.add(value);
    }

    private String format(double value) {
        return value == Math rint(value) ? String.format("%.0f", value)
: String.format("%.1f", value);
    }
}

```